

FACULDADE DE INFORMÁTICA E ADMINISTRAÇÃO PAULISTA
SISTEMAS DE INFORMAÇÃO

GABRIEL NOTARI

PRISCILA MANTOVANI

SOPHIE MOREAU

SMARTH HAS AI - FLORE APP

NETWORK MANAGEMENT NA SOCIEDADE 5.0

SÃO PAULO

2025

LINKS

Apresentação em vídeo: <https://youtu.be/sn-hsEqJ078>

Repositório: <https://github.com/primantovani/flore-app-flutter/tree/main>

Parte 1 — Versão Atual do Projeto

1.1 Visão Geral da Evolução

Desde a última entrega, o projeto Florê/Smart HAS evoluiu de uma proposta conceitual para uma aplicação mobile mais estruturada, com foco em experiência do usuário, inteligência artificial, logística e acompanhamento de entregas. Nesta nova fase, o sistema passa a integrar recursos que aproximam o aplicativo de uma solução real para gestão inteligente de produtos, armazenamento, rastreamento e comunicação com o usuário.

O Florê, que inicialmente estava mais concentrado na experiência de compra e curadoria de produtos, agora amplia sua proposta para incluir uma camada logística mais completa. Essa evolução permite que o usuário acompanhe pedidos, visualize pontos importantes no mapa, receba alertas sobre entregas e tenha uma experiência mais clara sobre o status dos produtos dentro da operação.

1.2 Nova Camada de Inteligência Artificial

A camada de inteligência artificial passa a atuar como apoio à tomada de decisão dentro do sistema. No contexto do Florê/Smart HAS, a IA pode ser aplicada para analisar dados de pedidos, comportamento do usuário, condições externas e status logístico, ajudando a tornar a experiência mais personalizada e eficiente.

Entre as possibilidades de aplicação da IA no projeto, destacam-se:

Recomendação inteligente de produtos com base no perfil e interesse do usuário.

Análise de produtos favoritos e sugestão de itens semelhantes.

Apoio à previsão de entrega, considerando status do pedido, localização e condições externas.

Identificação de possíveis atrasos a partir de dados climáticos e logísticos.

Geração de alertas automáticos para o usuário em situações relevantes, como item disponível, pedido em trânsito ou clima adverso.

Com isso, a IA não aparece apenas como um recurso visual ou complementar, mas como uma camada estratégica para melhorar a jornada do usuário e apoiar a operação logística.

1.3 Evolução da Experiência Mobile

Nesta fase, o aplicativo mobile passa a ter maior protagonismo no projeto. A experiência foi pensada para que o usuário consiga acompanhar sua jornada de forma simples, visual e intuitiva, sem depender apenas de códigos de rastreamento ou informações fragmentadas.

A proposta mobile considera telas voltadas para navegação, acompanhamento de pedidos, mapa de entregas, notificações e histórico de movimentações. A interface busca apresentar informações relevantes de forma objetiva, priorizando clareza, usabilidade e comunicação direta com o usuário.

A tela de rastreamento, por exemplo, representa uma das principais evoluções da experiência mobile, pois permite visualizar a localização do usuário, os pontos logísticos, os pedidos ativos e o impacto de fatores externos, como clima, na entrega.

1.4 Nova Camada Logística

A evolução logística do Florê/Smart HAS está relacionada à criação de uma estrutura capaz de acompanhar o ciclo de vida dos produtos dentro da operação. O sistema passa a considerar pontos como centro de armazenamento, parceiros logísticos, entregas em andamento e status dos pedidos.

Essa camada torna o projeto mais próximo de uma aplicação real de gestão logística, pois conecta o aplicativo mobile ao backend e aos serviços externos. Dessa forma, o usuário pode receber informações atualizadas sobre o andamento da entrega, enquanto a operação pode ser acompanhada de forma mais organizada.

A proposta também permite simular um modelo em que produtos são recebidos, armazenados, preparados e enviados ao consumidor final, criando uma experiência mais completa entre compra, armazenamento e entrega.

1.5 Relação com a Logística da Leroy Merlin Brasil

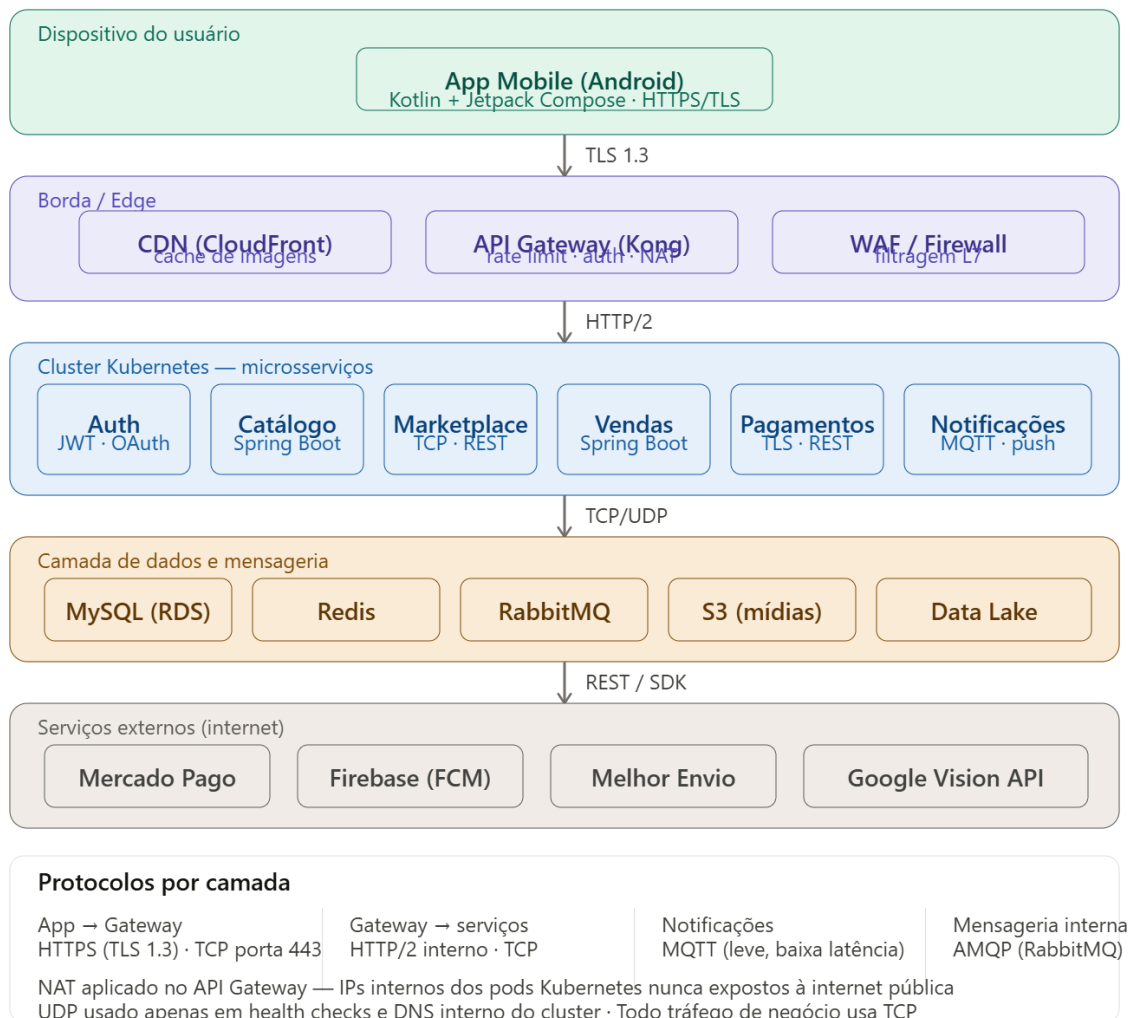
Considerando o contexto acadêmico da parceria com a Leroy Merlin Brasil, o Florê/Smart HAS pode ser relacionado a desafios reais de logística, como acompanhamento de entregas, visibilidade operacional, comunicação com o cliente e uso de dados para melhoria da experiência.

A aplicação proposta poderia contribuir para cenários em que o cliente precisa acompanhar produtos em trânsito, verificar status de entrega, receber avisos sobre atrasos ou consultar informações sobre pontos logísticos. Além disso, a integração com mapas, notificações e dados externos reforça a proposta de uma operação mais transparente e conectada.

Nesse sentido, o projeto dialoga com necessidades comuns do varejo e da logística, como previsibilidade, rastreamento, redução de dúvidas do cliente e melhor comunicação entre sistema, operação e usuário final.

Parte 2 — Arquitetura de Conectividade

2.1 — Topologia de Rede



2.2 — Roteamento, NAT e Protocolos de Transporte

A arquitetura de rede da Florê é estruturada em zonas de confiança progressiva. O tráfego originado no app mobile nunca alcança diretamente nenhum microsserviço, ele sempre passa pelo API Gateway, que funciona como único ponto de entrada público e aplica NAT, ocultando os endereços IP internos dos pods Kubernetes de qualquer exposição à internet.

O roteamento funciona da seguinte forma: o app resolve o domínio da Florê via DNS, obtém o IP do API Gateway, e estabelece uma conexão TLS 1.3 na porta 443. O Gateway então roteia a requisição internamente para o microsserviço correto com base no path da URL, `/auth/*` vai para o Auth Service, `/marketplace/*` vai para o Marketplace Service, usando HTTP/2 na rede interna do cluster, onde a sobrecarga de TLS não é necessária porque os pods se comunicam dentro de uma rede privada virtual gerenciada pelo Kubernetes.

O TCP é o protocolo de transporte dominante em toda a solução. Ele garante entrega ordenada e confiável das requisições REST, das queries ao MySQL, e das mensagens ao RabbitMQ, todos cenários onde a perda de pacotes seria inaceitável. O UDP aparece em dois contextos específicos e controlados: nas consultas DNS internas do cluster Kubernetes, onde a latência importa mais que a garantia de entrega, e nos health checks periódicos entre os componentes, onde um pacote perdido ocasionalmente é aceitável e a leveza do protocolo importa.

O MQTT é utilizado especificamente para o canal de notificações push. Esse protocolo foi escolhido por ser projetado para conexões instáveis e dispositivos com recursos limitados, exatamente o perfil de um smartphone em rede móvel. Diferentemente do HTTP, o MQTT mantém uma conexão persistente bidirecional entre o app e o broker, permitindo que o servidor envie mensagens ao cliente sem que o cliente precise fazer polling. O Firebase Cloud Messaging abstrai essa camada para o caso das notificações Android, usando MQTT internamente.

2.3 — Modelo Cliente-Servidor e Protocolos de Aplicação

A Florê segue o modelo cliente-servidor em todas as suas interações. O app mobile é sempre o cliente, nunca recebe conexões de entrada, apenas as inicia. Os microsserviços são os servidores, ficam aguardando requisições, processam e respondem. O único caso onde essa direção se inverte é nas notificações push, onde o servidor precisa iniciar a comunicação com o cliente, o que é resolvido pela conexão MQTT persistente mantida pelo Firebase.

O protocolo principal de comunicação entre o app e os serviços é o HTTP/1.1 sobre TLS, com respostas em JSON. A escolha por HTTP/1.1 em vez de HTTP/2 no lado do cliente reflete a compatibilidade com a maior gama possível de dispositivos Android, incluindo versões mais antigas do sistema que o app suporta a partir da API 21. Internamente, entre os microsserviços, HTTP/2 é utilizado plenamente, aproveitando o multiplexing de requisições sobre uma única conexão TCP para reduzir latência.

O AMQP é o protocolo usado pelo RabbitMQ para a comunicação assíncrona entre microsserviços. Quando o Sales Service confirma uma venda, ele publica uma mensagem em uma fila AMQP. O Notification Service consome essa fila de forma independente e despacha a notificação ao comprador e ao vendedor. Esse desacoplamento garante que uma lentidão no serviço de notificações não afete o tempo de resposta da confirmação de venda.

Parte 3 — Monitoramento e Gerenciamento

3.1 — Elementos que precisam ser monitorados

A infraestrutura da Florê possui três categorias distintas de elementos que precisam de monitoramento contínuo, cada uma com características e ferramentas específicas.

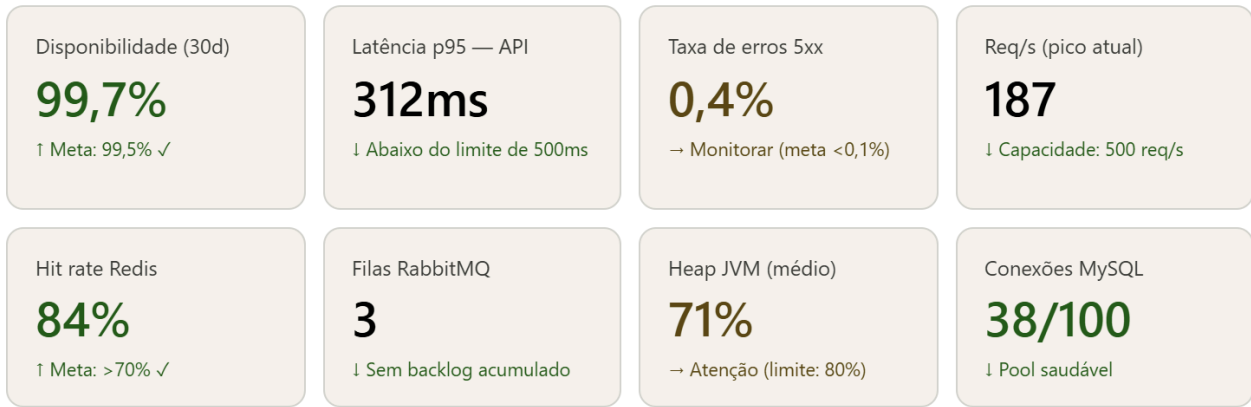
Os serviços de aplicação precisam ter sua disponibilidade, latência e taxa de erros monitorados continuamente. Isso inclui o tempo de resposta de cada endpoint REST, com atenção especial ao checkout e à autenticação, onde latência elevada impacta diretamente a conversão, a taxa de erros HTTP por serviço 4xx indicam problemas de uso, 5xx indicam falhas internas, e o tempo de processamento de mensagens nas filas do RabbitMQ, que indica se os consumidores estão acompanhando o ritmo de produção.

A infraestrutura de dados exige monitoramento de uso de conexões no pool do HikariCP esgotamento do pool é o sinal mais precoce de um problema de concorrência , utilização do heap JVM por pod pico acima de 80% do limite configurado antecede um OOMKill , e hit rate do Redis taxa abaixo de 70% indica que o cache está subdimensionado ou com TTL muito curto.

Os serviços externos merecem monitoramento independente porque estão fora do controle da equipe. A disponibilidade do gateway de pagamento deve ser verificada com uma chamada de health check a cada minuto, e qualquer degradação deve acionar o fallback automaticamente. O Firebase Cloud Messaging deve ter sua latência de delivery monitorada, notificações com mais de 30 segundos de atraso indicam congestionamento no serviço.

3.2 — Painel de NOC com KPIs Principais

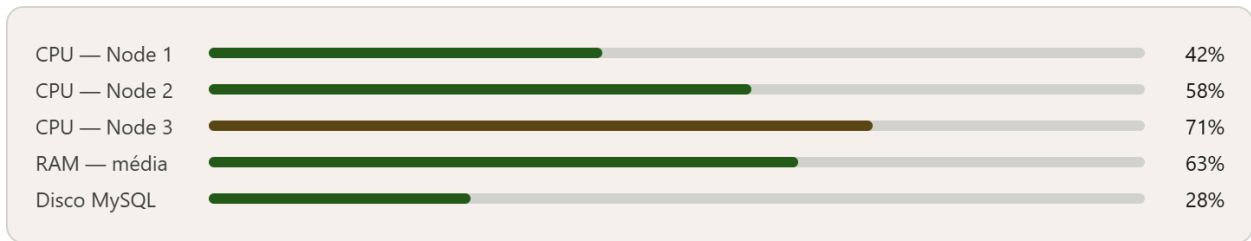
NOC — Florê · Painel de operações
Atualização simulada · Ambiente de referência



STATUS DOS MICROSERVIÇOS

Serviço	Latência p95	Erros 5m	Pods
● Auth Service	89ms	0	2/2
● Catalog Service	142ms	0	2/2
● Marketplace	312ms	4	3/3
● Sales Service	201ms	0	2/2
● Payment Service	487ms	2	2/2
● Notification Svc	54ms	0	1/1
● AI/ML Service	890ms	0	1/1

USO DE RECURSOS — NODES KUBERNETES



SLAS MONITORADOS

Disponibilidade da plataforma	Meta 99,5%	99,7% ✓
Carregamento do feed (<2s)	p95 atual	1,4s ✓
Resposta de suporte (<4h)	Média 30d	2,1h ✓
Latência checkout (<3s)	p95 atual	2,8s →
Entrega de notificação (<30s)	Mediana	4,2s ✓

3.3 — Princípios de Segurança, Escalabilidade e Resiliência

Segurança

A segurança da Florê é aplicada em camadas, de forma que o comprometimento de uma camada não implique o comprometimento das demais. Na camada de rede, todo tráfego externo é obrigatoriamente TLS 1.3, e o WAF filtra requisições maliciosas antes que alcancem qualquer serviço de aplicação. Na camada de aplicação, o API Gateway valida o token JWT em toda requisição autenticada antes de encaminhá-la ao microsserviço correspondente, garantindo que nenhum serviço interno precise reimplementar autenticação. Na camada de dados, todos os dados pessoais armazenados no MySQL são criptografados em repouso, e o acesso ao banco é permitido apenas por meio das credenciais gerenciadas pelo Kubernetes Secrets, nunca hardcodadas no código. Em conformidade com a LGPD, os dados de usuários inativos por mais de 24 meses são excluídos automaticamente por um job agendado, e o inventário de dados pessoais tratados pela plataforma é documentado como base para o Relatório de Impacto à Proteção de Dados.

Escalabilidade

A arquitetura da Florê escala de duas formas complementares. A escalabilidade horizontal é garantida pelo Horizontal Pod Autoscaler do Kubernetes, que adiciona réplicas de um microsserviço quando o uso de CPU ou o número de requisições por segundo ultrapassa o limiar configurado, fixado em 70% de CPU para o MVP. Cada microsserviço escala de forma independente, de modo que um pico de tráfego no Marketplace Service não provoca alocação desnecessária de recursos no Auth Service ou no Notification Service. A escalabilidade vertical é reservada para o banco de dados, onde a instância MySQL no RDS pode ter sua classe de hardware ampliada com mínima interrupção, independentemente da escala dos serviços de aplicação.

Resiliência

A resiliência é tratada como princípio de design, não como característica adicionada após a implementação. Todo microsserviço implementa circuit breaker, com o padrão Resilience4j no ecossistema Spring Boot, que interrompe chamadas a um serviço downstream quando a taxa de falhas ultrapassa 50% em uma janela de 10 segundos, evitando que uma falha pontual se propague em cascata por toda a cadeia de serviços. As operações assíncronas críticas, como o disparo de notificações e o retreinamento da IA, são gerenciadas via RabbitMQ com dead letter queues, garantindo que nenhuma mensagem seja perdida em caso de falha de um consumidor. O banco de dados MySQL é provisionado em modo Multi-AZ na fase escalável, com failover automático para a réplica em caso de indisponibilidade da instância primária. O objetivo final é que

nenhuma falha isolada de um pod, de um banco, de um serviço externo resulte em indisponibilidade total da plataforma para o usuário.

Parte 4 — Implementação Mobile com Flutter

4.1 Visão Geral da Implementação Mobile

A implementação mobile do Florê/Smart HAS foi desenvolvida com Flutter, permitindo a criação de uma aplicação multiplataforma com uma única base de código. A escolha do Flutter contribui para maior agilidade no desenvolvimento, padronização visual e facilidade na criação de interfaces modernas e responsivas.

O aplicativo foi estruturado com foco na experiência do usuário, reunindo funcionalidades como visualização de pedidos, rastreamento em mapa, notificações, dados climáticos e integração com serviços externos. A proposta é oferecer uma navegação simples, objetiva e visualmente organizada, aproximando o usuário da operação logística do sistema.

4.2 Principais Telas do Aplicativo

As principais telas previstas para o aplicativo Florê/Smart HAS são:

Tela Inicial/Home

Apresenta uma visão geral da aplicação, destacando informações principais do usuário, pedidos ativos, atalhos para rastreamento e notificações importantes. Essa tela funciona como ponto de entrada da experiência mobile.

Tela de Pedidos

Exibe a lista de pedidos do usuário, com informações como nome do item, status atual, previsão de entrega e percentual de progresso. A tela permite que o usuário acompanhe de forma centralizada todos os pedidos vinculados à sua conta.

Tela de Rastreamento/Mapa

Mostra o mapa interativo com a localização do usuário, centro de armazenamento, entregas em andamento e parceiros logísticos. Essa tela é fundamental para o tracking inteligente, pois transforma dados logísticos em uma visualização geográfica mais clara.

Tela de Detalhes do Pedido

Apresenta informações completas sobre um pedido específico, incluindo origem, destino, status, previsão de entrega e timeline dos eventos. Essa tela ajuda o usuário a compreender cada etapa da movimentação do produto.

Tela de Notificações

Reúne alertas enviados pelo sistema, como pedido em trânsito, item recebido no armazém, favorito disponível e alerta climático. Também pode ser utilizada para demonstrar o funcionamento das notificações push via Firebase Cloud Messaging.

Tela de Perfil/Configurações

Permite ao usuário visualizar seus dados, ajustar preferências, permissões de localização e configurações de notificação.

4.3 Organização Visual e Funcional

A estrutura visual do aplicativo foi pensada para priorizar clareza, hierarquia da informação e facilidade de navegação. As informações mais importantes, como status do pedido e previsão de entrega, devem aparecer em destaque, enquanto detalhes complementares podem ser acessados em cards, listas ou bottom sheets.

A organização funcional segue uma lógica de jornada do usuário:

O usuário acessa o app.

Visualiza seus pedidos ativos.

Seleciona um pedido.

Acompanha o pedido no mapa.

Recebe notificações sobre eventos relevantes.

Consulta detalhes e histórico quando necessário.

Essa estrutura favorece uma experiência fluida, em que o usuário não precisa procurar informações em diferentes canais ou interpretar dados técnicos de rastreamento.

4.4 Componentização com Widgets

No Flutter, a interface foi organizada por meio de widgets reutilizáveis. Essa abordagem facilita a manutenção do código, evita repetição e permite que os componentes sejam reaproveitados em diferentes telas.

Entre os principais componentes previstos ou utilizados no app, destacam-se:

OrderCard: exibe resumo de um pedido, com nome, status e progresso.

WeatherCard: apresenta dados climáticos e impacto na entrega.

MapMarker: representa pontos relevantes no mapa, como usuário, armazém, entrega ou parceiro.

TimelineItem: mostra cada etapa da movimentação do pedido.

NotificationButton: permite simular ou disparar notificações na tela de demonstração.

StatusBadge: indica visualmente o status atual do pedido.

BottomSheetDetails: exibe detalhes completos ao tocar em um pedido ou marcador.

Essa componentização torna o projeto mais organizado e facilita futuras melhorias na interface.

4.5 Gerenciamento de Estado

O gerenciamento de estado utilizado no projeto segue o padrão Provider, com classes baseadas em ChangeNotifier. Esse padrão permite separar a lógica de negócio da interface, mantendo o código mais limpo e organizado.

No caso da tela de mapa, por exemplo, o MapProvider é responsável por controlar informações como localização do usuário, carregamento de dados climáticos, pontos no mapa e pedidos ativos. Quando os dados são atualizados, o provider notifica os widgets dependentes, e a interface é reconstruída automaticamente.

Esse modelo facilita o controle de estados como:

Carregamento inicial da tela.

Sucesso na obtenção de dados.

Falha ao buscar localização ou serviços externos.

Atualização dos pedidos e pontos no mapa.

Exibição de alertas e notificações.

4.6 Justificativa do Uso do Provider

O Provider foi escolhido por ser uma solução simples, produtiva e adequada ao nível atual do projeto. Ele permite organizar o estado da aplicação sem adicionar complexidade excessiva, sendo suficiente para controlar telas, serviços e dados assíncronos.

Além disso, o Provider facilita a integração entre a interface e os serviços do aplicativo, como `DeliveryService`, `WeatherService` e `FCMService`. Dessa forma, a aplicação mantém uma separação clara entre tela, estado e comunicação com APIs externas.

Para versões futuras, caso o projeto cresça em complexidade, também seria possível migrar para uma arquitetura mais robusta, como BLoC ou Riverpod. Porém, para esta fase acadêmica, o Provider atende bem aos objetivos de clareza, produtividade e organização.

Parte 5 — Mapas e Geolocalização

5.1 Visão Geral

A Parte 5 do projeto Florê implementa a funcionalidade de mapas e geolocalização no aplicativo mobile, utilizando o Google Maps SDK para Flutter. O objetivo é fornecer ao usuário uma visão geográfica completa de sua jornada de entrega, exibindo em tempo real a localização do usuário, o centro de armazenamento Florê, as entregas em andamento e os parceiros logísticos.

A integração com geolocalização é um componente central da proposta de tracking inteligente do Florê, conectando a camada visual do aplicativo com os dados logísticos em tempo real retornados pelo back-end.

5.2 Tela de Rastreamento — MapScreen

A tela principal de mapas foi implementada no arquivo `lib/screens/map_screen.dart` e é composta pelos seguintes elementos:

Componente	Descrição
Google Maps	Mapa interativo centralizado na localização do usuário com zoom 13
WeatherCard	Card no topo exibindo temperatura, vento e impacto na entrega
Marcadores	Pontos coloridos no mapa para cada tipo de localização
Lista de Pedidos	Carrossel horizontal com pedidos ativos e progresso
Bottom Sheet	Painel de detalhe ao tocar em um pedido com timeline completa

5.3 Tipos de Marcadores no Mapa

Quatro categorias de pontos são exibidos no mapa, cada um com cor distinta para facilitar a identificação visual:

- Azul — Localização do usuário em tempo real, obtida via geolocator
- Verde — Centro de armazenamento Florê em São Paulo, ponto central da operação logística
- Laranja — Entregas em andamento, com informações de status e previsão de chegada
- Roxo — Parceiros logísticos como Correios e Jadlog, pontos de apoio à distribuição

5.4 Geolocalização e Permissões

A obtenção da localização do usuário é feita através do pacote `geolocator`, com o seguinte fluxo de permissões implementado no `MapProvider`:

- Verifica se o serviço de localização está habilitado no dispositivo
- Verifica o status atual da permissão (negada, negada permanentemente ou concedida)
- Solicita permissão ao usuário caso ainda não tenha sido concedida
- Obtém posição com precisão alta (LocationAccuracy.high)
- Em caso de falha, centraliza o mapa em São Paulo como fallback

As permissões necessárias foram declaradas no AndroidManifest.xml:

ACCESS_FINE_LOCATION, ACCESS_COARSE_LOCATION

5.5 Contribuição da Geolocalização para o Tracking Inteligente

A geolocalização integrada ao mapa contribui para o tracking inteligente do Florê de três formas principais:

5.5.1 Contextualização da Entrega

Ao conhecer a posição do usuário e a posição da entrega em tempo real, o sistema calcula a distância restante e exibe a informação de forma clara na interface, eliminando a necessidade de o usuário interpretar códigos de rastreamento.

5.5.2 Integração com Dados Climáticos

A localização do usuário é enviada como parâmetro para o serviço de clima (Open-Meteo), que retorna as condições meteorológicas da região. Esses dados alimentam o WeatherCard exibido no topo da tela de mapa, indicando se as condições são favoráveis ou adversas para entrega. Em casos de chuva intensa ou vento forte, o sistema exibe um alerta visual e pode disparar uma notificação push via Firebase Cloud Messaging.

5.5.3 Foco Dinâmico no Mapa

Ao tocar em um pedido na lista horizontal, o mapa anima sua câmera para centralizar no ponto de entrega correspondente, permitindo ao usuário visualizar a rota e o contexto geográfico de forma intuitiva, sem necessidade de navegar manualmente pelo mapa.

5.6 Gerenciamento de Estado — MapProvider

O estado da tela de mapa é gerenciado pelo MapProvider, uma classe ChangeNotifier registrada no Provider. Ao ser inicializado, o provider executa em paralelo três chamadas assíncronas:

- Obtenção da localização do usuário via geolocator
- Busca de dados climáticos via WeatherService (Open-Meteo)
- Busca de pontos do mapa e pedidos via DeliveryService (flore-back)

O uso de Future.wait garante que todas as operações sejam executadas simultaneamente, reduzindo o tempo de carregamento. Ao completar, o provider notifica os widgets dependentes via notifyListeners(), atualizando o mapa, o card de clima e a lista de pedidos de uma só vez.

Parte 6 — Integração com Firebase e Web Services

6.1 Visão Geral das Integrações

A Parte 6 implementa a integração do aplicativo Florê com dois serviços web externos e com o Firebase Cloud Messaging para notificações push. O objetivo é enriquecer a experiência do usuário com dados em tempo real e comunicação proativa.

Serviço	Tipo
Open-Meteo API	API REST gratuita — dados climáticos em tempo real
Florê Back-end	API REST própria — pedidos, entregas e pontos do mapa
Firebase Cloud Messaging	Push Notifications — alertas e avisos do sistema

6.2 Serviço 1 — Open-Meteo API

6.2.1 Descrição do Serviço

O Open-Meteo é uma API meteorológica gratuita e de código aberto, disponível em api.open-meteo.com. Não requer chave de API, sendo ideal para o contexto acadêmico do projeto. A integração foi implementada no arquivo `lib/services/weather_service.dart`.

6.2.2 Dados Retornados

A API retorna os seguintes dados para a localização do usuário:

- `temperature_2m` — Temperatura atual em graus Celsius
- `wind_speed_10m` — Velocidade do vento em km/h
- `weather_code` — Código WMO indicando a condição climática (0=céu limpo, 61-67=chuva, 80-82=pancadas)

6.2.3 Aplicação no Sistema

Os dados climáticos são exibidos no WeatherCard da tela de mapa e utilizados para determinar se as condições são favoráveis para entrega. Quando o código indica chuva intensa (código ≥ 60) ou vento acima de 50 km/h, o sistema exibe alerta visual e pode disparar notificação push via FCM.

6.3 Serviço 2 — Florê Back-end

6.3.1 Descrição do Serviço

O back-end do Florê é uma API REST desenvolvida em Java com Spring Boot, hospedada no repositório github.com/Gabrielnotari/flore-back. A integração foi implementada no arquivo `lib/services/delivery_service.dart`, com fallback automático para dados simulados quando o servidor não está disponível.

6.3.2 Endpoints Utilizados

Endpoint	Dados Retornados
GET /api/orders	Lista de pedidos ativos do usuário com status, progresso e timeline

GET /api/map-points

Pontos geográficos para exibição no mapa (armazéns, entregas, parceiros)

6.3.3 Estrutura dos Dados

Os pedidos retornados pela API seguem o modelo `DeliveryOrder`, contendo: identificador do pedido, nome do item, status atual, origem, destino, previsão de entrega, percentual de progresso e lista de eventos da timeline.

Os pontos do mapa seguem o modelo `DeliveryPoint`, contendo: identificador, rótulo, tipo (warehouse/delivery/partner), coordenadas geográficas, status e previsão de chegada.

6.4 Firebase Cloud Messaging — Notificações Push

6.4.1 Configuração

A integração com o Firebase Cloud Messaging foi implementada no arquivo `lib/services/fcm_service.dart`. O serviço inicializa o Firebase no startup do app, solicita permissões de notificação ao usuário e configura listeners para mensagens recebidas em foreground e background.

A configuração do canal de notificações Android foi realizada com as seguintes propriedades:

- ID do canal: `flore_alerts`
- Nome: Alertas Florê
- Importância: `High` — garante exibição imediata com som e vibração

6.4.2 Tipos de Notificações Implementadas

Quatro tipos de notificação push foram implementados e podem ser disparados via tela de demonstração:

Tipo

Gatilho

Entrega em Trânsito

Pedido saiu para entrega — exhibe previsão de chegada

Item no Armazém	Item chegou ao centro Florê — confirma recebimento
Favorito Disponível	Peça curtida voltou ao catálogo — incentiva reengajamento
Alerta Climático	Chuva intensa detectada — informa possível atraso na entrega

6.4.3 Fluxo de Funcionamento

O fluxo completo de uma notificação push no sistema Florê funciona da seguinte forma:

- 1. O servidor Florê (back-end Spring Boot) detecta um evento relevante (pedido saiu para entrega, clima adverso)
- 2. O servidor envia o payload FCM para o Firebase Admin SDK
- 3. O Firebase entrega a mensagem ao dispositivo do usuário via protocolo FCM
- 4. Se o app está em foreground: `FirebaseMessaging.onMessage` captura a mensagem e exibe notificação local via `flutter_local_notifications`
- 5. Se o app está em background: o handler `_firebaseMessagingBackgroundHandler` processa a mensagem
- 6. Ao tocar na notificação: `FirebaseMessaging.onMessageOpenedApp` permite navegar para a tela relevante

6.4.4 Tela de Demonstração

A tela `lib/screens/notification_screen.dart` foi implementada para demonstrar o funcionamento das notificações push sem necessidade de um servidor FCM real em produção. Ela exibe o status de inicialização do Firebase, botões para disparar cada tipo de notificação e um histórico das notificações disparadas na sessão atual.

Parte 7 — Comparações entre Flutter e Kotlin

7.1 Visão Geral da Comparação

Durante o desenvolvimento do projeto, foi possível comparar a experiência anterior com Kotlin e a nova implementação utilizando Flutter. As duas tecnologias permitem criar aplicações mobile, mas apresentam diferenças importantes em produtividade, curva de aprendizado, organização visual e integração com recursos externos.

Kotlin é uma linguagem muito utilizada no desenvolvimento Android nativo, oferecendo alto desempenho e integração direta com o ecossistema Android. Já o Flutter permite desenvolver aplicações multiplataforma com uma única base de código, utilizando a linguagem Dart e uma estrutura baseada em widgets.

No contexto do Florê/Smart HAS, o Flutter se mostrou mais produtivo para a construção rápida das telas, organização visual e integração de diferentes recursos em uma aplicação mobile.

7.2 Pontos Positivos do Flutter

O principal ponto positivo do Flutter foi a agilidade no desenvolvimento das interfaces. Como a construção visual é feita por widgets, tornou-se mais simples criar telas reutilizáveis, organizar componentes e manter uma identidade visual consistente no aplicativo.

Outro ponto positivo foi a possibilidade de desenvolver uma aplicação com potencial multiplataforma, reduzindo a necessidade de criar versões separadas para Android e iOS. Isso é importante para um projeto acadêmico, pois permite focar mais na experiência, na arquitetura e nas funcionalidades do sistema.

Também se destacou a facilidade de integração com pacotes prontos, como Google Maps, geolocator, Firebase Messaging e consumo de APIs REST. Esses recursos ajudaram a acelerar a implementação de funcionalidades importantes, como mapa, localização, dados climáticos e notificações.

7.3 Pontos Negativos ou Desafios do Flutter

Apesar das vantagens, o Flutter também apresentou alguns desafios. A estrutura baseada em widgets pode se tornar extensa quando a tela possui muitos elementos visuais, exigindo boa organização do código e separação em componentes menores.

Outro desafio foi compreender o gerenciamento de estado. Mesmo utilizando Provider, foi necessário organizar bem a responsabilidade de cada classe para evitar que a lógica da aplicação ficasse misturada com a interface.

Além disso, algumas configurações nativas, como permissões Android, Firebase e notificações em background, ainda exigem ajustes em arquivos específicos da

plataforma, o que pode ser mais trabalhoso para quem espera trabalhar apenas dentro do código Dart.

7.4 Pontos Positivos do Kotlin

O Kotlin apresenta como principal vantagem a integração direta com o Android. Por ser uma tecnologia nativa, ele oferece maior controle sobre recursos específicos do sistema operacional, permissões, ciclo de vida da aplicação e componentes próprios da plataforma.

Também é uma linguagem robusta, moderna e bastante utilizada no mercado para desenvolvimento Android. Em projetos que exigem alto nível de personalização nativa ou uso intenso de recursos específicos do Android, Kotlin pode ser uma escolha muito forte.

7.5 Pontos Negativos ou Desafios do Kotlin

Em comparação com Flutter, o desenvolvimento em Kotlin pode ser mais demorado quando o objetivo é criar interfaces rapidamente ou manter uma experiência visual padronizada em diferentes plataformas.

Outro ponto é que, para desenvolver também para iOS, seria necessário criar outra aplicação ou utilizar outras estratégias de compartilhamento de código. Isso aumenta o esforço de desenvolvimento e manutenção.

Para o contexto acadêmico do Florê/Smart HAS, Kotlin exigiria mais tempo para alcançar o mesmo nível de prototipação visual e integração demonstrada com Flutter.

7.6 Processo Mais Simples e Produtivo

O processo com Flutter foi mais simples e produtivo para esta fase do projeto. A criação das telas, a componentização por widgets e a integração com pacotes externos tornaram o desenvolvimento mais ágil.

A estrutura do Flutter também facilitou a construção de uma experiência mobile mais visual, com cards, mapas, listas horizontais, bottom sheets e notificações. Esses elementos são importantes para apresentar o Florê/Smart HAS como uma solução moderna e funcional.

7.7 Processo Mais Desafiador

O ponto mais desafiador no Flutter foi organizar corretamente o estado da aplicação e integrar recursos que dependem de permissões, serviços externos e configuração

nativa. Funcionalidades como localização em tempo real, notificações push e comunicação com APIs exigiram atenção especial.

Mesmo assim, o resultado foi positivo, pois a tecnologia permitiu conectar diferentes partes do sistema em uma única experiência mobile: mapa, pedidos, clima, notificações e backend.

7.8 Conclusão da Comparação

A comparação entre Kotlin e Flutter mostrou que Kotlin é uma opção forte para desenvolvimento Android nativo, principalmente quando há necessidade de controle profundo da plataforma. No entanto, para o Florê/Smart HAS, Flutter se mostrou mais adequado por permitir maior produtividade, construção rápida de interfaces, reutilização de componentes e integração com serviços externos.

Dessa forma, a escolha do Flutter contribuiu diretamente para a evolução do projeto nesta fase, permitindo que o aplicativo deixasse de ser apenas uma proposta conceitual e passasse a representar uma experiência mobile mais completa, integrada e próxima de uma aplicação real.